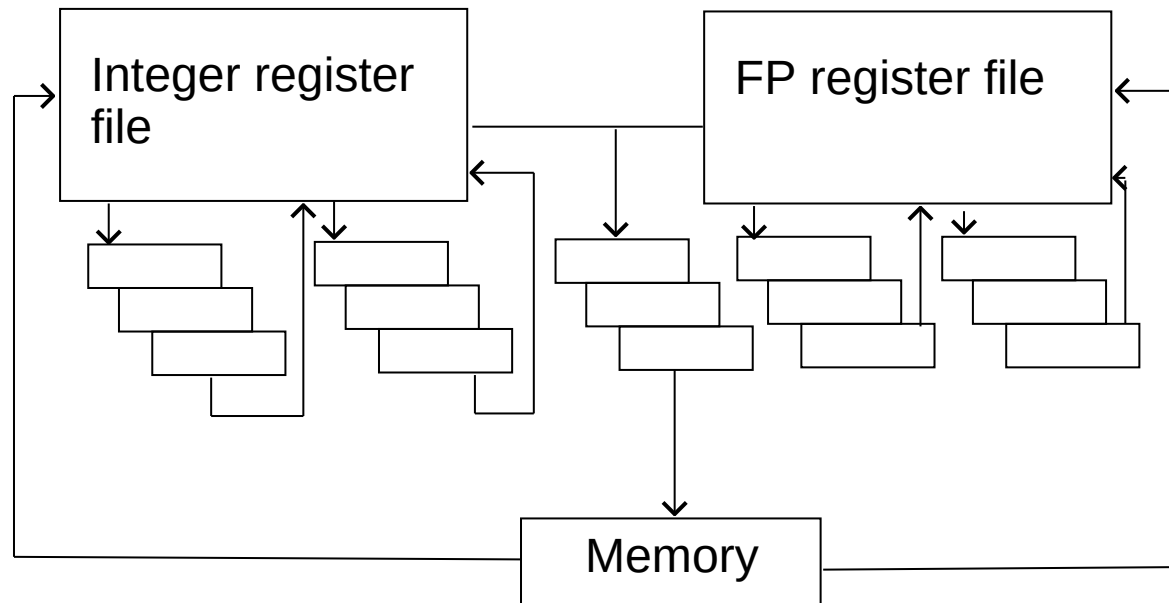


Introduction to Superscalar Architectures

The term **superscalar** literally refers to a machine that improves on the performance of scalar instructions.

In practice, whereas RISC architectures try to achieve **one instruction completion per cycle**, superscalar machines the ability to achieve **multiple instruction completions per cycle**. In other words, on average, instructions take **fractions** of a clock cycle to complete.

The general organization of a superscalar processor is:



By creating **multiple, pipelined, functional units**, the processor can have several instructions executing simultaneously.

Limitations of superscalar processors

Superscalar systems depend to a high degree on **parallel execution** of instructions. Instead of executing instructions one after the other in the standard Von Neumann approach, multiple instructions are fetched, decoded and executed simultaneously. There are two fundamental limitations to this approach:

Instruction-level parallelism The degree to which instructions can be executed in parallel. We will spend a lot of time looking at the kinds of dependencies that can exist between instructions in a program.

Machine-level parallelism Measures the **ability of the hardware** to take advantage of instruction-level parallelism. For example, instructions in a program may exist that can be parallelized, but if the hardware does not exist to execute them in parallel, this does not help.

In general, instruction-level parallelism is the more fundamental limitation. You can always get more machine-level parallelism by throwing hardware at the problem.

Instruction-level parallelism

There are five fundamental limitations to instruction level parallelism:

- “True” data dependency
- Procedural dependency
- Resource conflict
- Output dependency
- Antidependency

No matter how much parallelism the processor performs, the result must be the same as if there were none.

True data dependency (write-read)

This is the type of data dependency that we have already seen. Consider this sequence:

I1: ADD R1, R1, R2

I2: MOV R3, R1

This would not be a problem with RISC execution. However, we can **not** execute these instructions in parallel. I2 needs to wait for the result of I1 before it can execute. If they are fetched simultaneously, I2 will have to stall waiting for I1.

Procedural dependency

The first type of procedural dependency is one we have already seen: the branch penalty. With a superscalar processor, branch penalties will be magnified by the amount of parallelism.

There is a second type of procedural dependency that relates to instruction size. If instructions can have variable size (as we saw with Pentium), the instruction I1 must be decoded before instruction I2 can even be fetched! For this reason, superscalar techniques are easier to apply to RISC-like architectures, with a fixed instruction size.

Resource conflict

Resource conflicts occur when two instructions need the same resource (ALU, memory bus, register-file, etc) simultaneously.

From a theoretical standpoint, this is not really a limitation.
We just throw hardware at the problem until it goes away.

Instruction issue policy

To understand the remaining types of dependencies, we need to understand instruction issue policies. There are three fundamental choices:

In-order issue, in-order completion Instructions are issued in the order they appear, and must be allowed to complete in correct sequence. Not even simple RISC machines are this strict.

In order issue, out-of-order completion Instructions are issued in the order they appear, but they can complete at their convenience. If I1 takes longer to complete than I2, I2 is allowed to complete first, then I3 can be fetched. Can leads to **output dependencies**.

Out-of-order issue, out-of-order completion In this approach, the processor can “look ahead” to find more

instructions to fetch, even if dependencies exist. Can lead to an **antidependency**

Output dependency (write-write)

Consider the following fragment:

I1: $R3 \leftarrow R3 \times R5$

I2: $R4 \leftarrow R3 + 1$

I3: $R3 \leftarrow R5 + 1$

With out of order completion, I3 could finish **before** I1. This will lead to an incorrect result, since R3 will have the wrong value at the end of the program.

Antidependency

Consider our sequence again:

I1: $R3 \leftarrow R3 \times R5$

I2: $R4 \leftarrow R3 + 1$

I3: $R3 \leftarrow R5 + 1$

Suppose I3 completes before I2 has fetched its operands. Then R4 will get the wrong value because R3 has updated its operand. This is like a data dependency **in reverse**.

Instruction Windowing

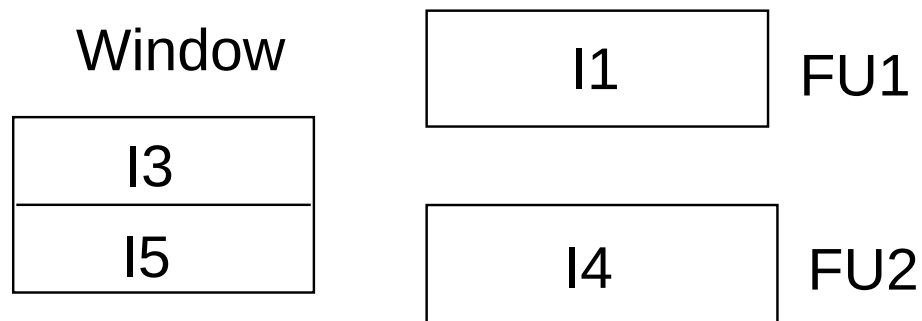
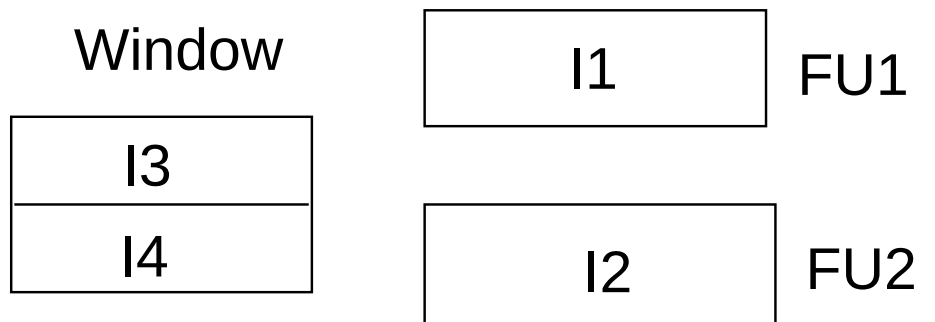
With **in-order issue**, the processor will decode instructions (determine source and destination operands) until it finds a **data dependency** or **resource conflict**. At that point, it must stall the pipeline to wait until the conflict has been resolved.

The goal of **out-of-order issue** is to “look ahead” to find future instructions that are independent and can be executed.

The usual technique for out-of-order issue is called **instruction windowing**. Instructions that have been **decoded but not executed** are placed into a buffer called the **instruction window**. Fetching and decoding can proceed (ie: the processor does not stall) as long as the instruction window is not full.

Instructions move from the window into the execution phase when the appropriate functional unit becomes free, if no dependencies exist.

The figure shows an example two instruction window. I1 takes three execute cycles but I2, I3 and I4 all take one cycle and I3 depends on the result of I1. I4, therefore, can be issued before I3.



Register Renaming

“True data dependencies” are, in some sense, unavoidable, since they reflect the logical flow of the program.

Output dependencies and antidependencies, however, can be viewed as **artifacts** of out-of-order instruction execution and issue. As a result, they can be avoided by intelligent hardware, through a scheme known as **register renaming**.

The key to register renaming is that output dependencies and antidependencies are **resource conflicts**. They exist because multiple instructions are trying to write to the same location. The way to solve resource conflicts, as we have seen, is to throw hardware at them.

Therefore, in a **register renaming** scheme, every time an instruction writes to a (logical) register, we will **dynamically allocate** a physical register to hold that value. The difficulty is that subsequent accesses to that value will have to access the right register.

Here is an example. Letters have been added to show the different “meanings” of the registers:

I1: $R3(B) \leftarrow R3(A) \text{ op } R5(A)$

I2: $R4(B) \leftarrow R3(B) + 1$

I3: $R3(C) \leftarrow R5(\textcolor{red}{A}) + 1$ Re-assignment avoids dependency

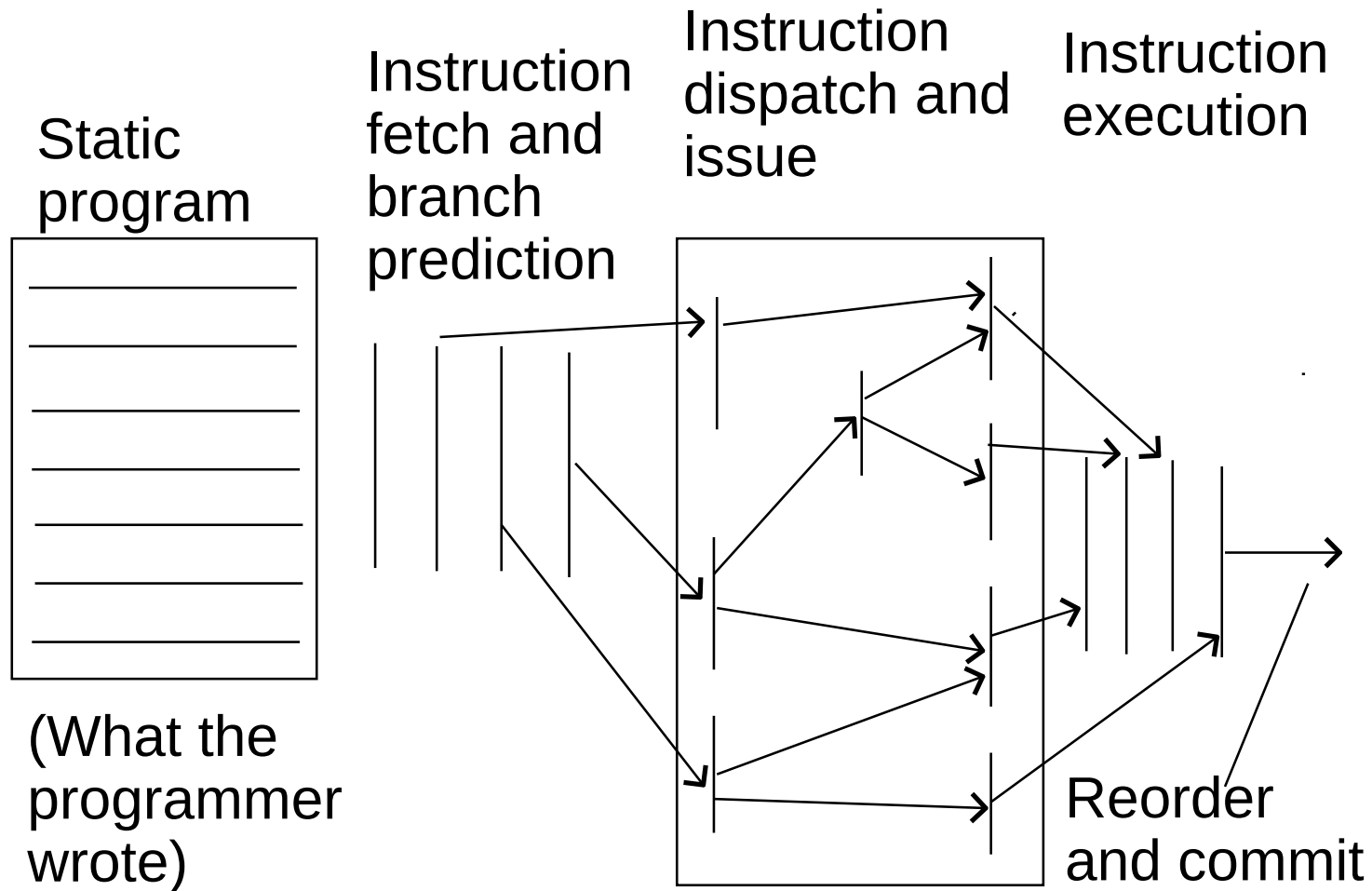
I4: $R7(B) \leftarrow R3(C) \text{ op } R4(B)$ True data dependency

Once all the instructions that refer to a given register are through the pipeline, we can re-use that register.

Research indicates the following two clues about implementation:

- It is not worth adding functional units without also using register renaming. Output dependencies and antidependencies are too common, and will limit the possible speedup.
- With register renaming, long instruction windows have significant effects. The further into the future the processor looks, the less likely “true” data dependencies will be.

A conceptual view of a superscalar processor:



Instruction fetch A dynamic stream of instructions that includes branch prediction.

Instruction dispatch/issue Instructions are placed into the instruction window and executed in an order dictated by **real dependencies** and **hardware conflicts**.

Instruction reorder/commit Results are only written back to “permanent” register storage *if they really would have happened*. If the instruction was on a predicted, but not taken branch, it must be discarded.